Secure Cloud-Based Intelligence Monitoring System For Anomaly Detection

Project Workbook

By

Devkumar Divyeshkumar Ansodariya
Kumar Harsh
Cameron Lee

10/02/2025

Advisor: Dr. Zeyu Gao

[

The **Project Workbook** is used to collect research, proposals, requirements, architecture, design, implementation, and project planning information. This assignment is not intended to be used as a writing assignment but as a technical assignment that collects a verbose amount of information about your project. You will ultimately select a subset of the information from this workbook to include in your Project Report. The Project Report will require strict attention to the detail required for a writing assignment.

The information collected in this workbook includes a problem description, a state-of-the-art description, literature search, project justification, requirements, architecture, design (including tradeoffs, UML artifacts, UI mockups, and database design), QA, performance validation planning, implementation/prototype development, project management planning (task assignment and schedule), and conclusions. This information is organized in nine chapters of this document. You can add additional chapters if you think your project has other information that needs to be collected.

Information in this document will be collected and evaluated across two assignments in CMPE 295A (referred to in this document as Workbook Assignment 1 and 2). The chapter descriptions below indicate which of the chapters are evaluated in each of the workbook assignments. You should continue to use this workbook as a collection point for information related to your project until your project is completed.

All members of your project team should contribute to this document. Each member of your project team should identify their individual contributions in this document.

]

Chapter 1. Literature Search, State of the Art

The Literature Search integrates the scholarly development of Video Anomaly Detection (VAD), while the State-of-the-Art Summary summarizes the nascent industry trends and technologies.

Literature Search

What are we searching for?:

We are looking for papers and journals that focus on the detection of certain anomalous events(fires, traffic accidents, criminal behavior, wild animals), improvements in performance of classification based on camera-based surveillance, improving image quality and alternative methods of detection, and cloud computing frameworks for surveillance.

Where are we searching:

- SJSU's Library with OneSearch

Key Words and Phrases: *surveillance, anomaly detection, frame, intelligent, computer vision, cloud computing, edge cloud, YOLO, spatial domain, temporal domain*

1. **Early Approaches to Video Anomaly Detection**

   The early VAD work was statistical model and motion analysis-based to define normal behavior and identify anomalies. Background subtraction, for example, used Gaussian Mixture Models (GMMs) and optical flow for movement vector detection. Both techniques were effective in extracting motion and trajectory-based outliers, such as automobiles in walk-through areas (Zhang et al., 2020). They struggled with appearance-based outliers (e.g., fire) and complex, multi-object scenarios.

2. **Object-Centric Detection with YOLO**

   Deep learning detectors in real-time changed the game of anomaly detection. The series of YOLO models shifted scene-level analysis to object-level detection and classification of individuals, vehicles, and smoke. Recent research emphasizes the effectiveness of YOLO in human behavior anomaly detection, vehicle accidents, and fire (Ganagavalli & Santhi, 2024). The latest version of the model, YOLOv8, is applied in this project, which

outperforms the previous versions with anchor-free detection and enhanced efficiency. While robust, YOLO handles frames sequentially, limiting its ability to capture temporal dynamics.

3. **Temporal and Contextual Models**

For time-critical anomalies, researchers suggested models learning temporal and spatial patterns:

- 3D-CNNs generalize 2D convolutions to the spatial-temporal domain, extracting appearance and motion simultaneously (Maqsood et al., 2021).
- LSTMs and other recurrent models track sequences, capable of distinguishing between deliberate action and accidents.
- Graph Neural Networks (GNNs) learn the interrelations among multiple objects (e.g., crowds of people), enabling detection of unusual interactions (Chen & Wu, 2024).

These methods provide contextual understanding that object detectors lack.

New Research Directions

Efficiency, flexibility, and generalizability are the targets in new work:

- Edge-Cloud Hybrid Architectures perform shallow inference on the edge and offload computationally heavy workloads to the cloud, reducing latency as well as bandwidth needs (Hussain et al., 2025).
- Self-Supervised Learning is surmounting the absence of labeled examples of anomalies by learning on proxy tasks (i.e., predicting frame order), and thus models are able to establish a baseline of "normalcy."
- Open-Set and Language-Guided Detection extends models from bounded anomaly classes, such that novel events can be identified and dynamic definitions of anomalies are achievable via natural language descriptions.

Taken together, the literature is biased toward multi-stage VAD systems that employ quick object detection (YOLO), temporal modeling (3D-CNN or LSTM), and adaptive methods (self-supervised or open-set recognition). This tiered approach directly impacts the proposed architecture of this project.

State-of-The-Art Summary

Industry is looking for scalable cloud-based surveillance solutions with integrated AI analytics as built-in features.

- Cloud-Native Platforms (VSaaS): Companies such as Eagle Eye Networks and Avigilon Alta are leading the transition from on-premise recorders to cloud-based, hardware-agnostic platforms. These platforms are integrated into other security solutions and are poised to realize a market growth from USD 3.9B in 2024 to USD 12B in 2030.

- AI-Powered Analytics: Existing systems possess object recognition, facial analysis, and behavior analytics onboard. The shift is from reactive surveillance to proactive security, with systems able to predict or prevent incidents.

- High-Performance SDKs and Toolkits: Frameworks like NVIDIA DeepStream SDK and managed cloud platforms (AWS SageMaker, Azure AI) take care of infrastructure complexities like video ingestion, decoding, batching, and scaling. This allows small teams to focus on building complex anomaly detection models rather than low-level engineering.

References

[1] Ganagavalli, K., & Santhi, V. (2024). YOLO-based anomaly activity detection system for human behavior analysis and crime mitigation. Signal, Image and Video Processing, 18(Suppl 1), 417–427.

The authors of this paper employed an improvised YOLO object detection algorithm, fine-tuned with hyperparameters for improved performance across 14 categories of abnormal activities. A custom dataset containing various criminal behaviors was used for training. The system classifies scenes as normal or abnormal based on visual activity, using CNN-based feature extraction and video processing to enable real-time detection.

[2] Zhang, X., Yang, S., Zhang, J., & Zhang, W. (2020). Video anomaly detection and localization using motion-field shape description and homogeneity testing. Pattern Recognition, 105, Article 107394.

Using video frames and object trajectories to calculate the probability of an abnormality in a crowd of people, this concept allows for the predictive identification of abnormal behavior using fewer frames. Frames from video feeds were analyzed based on pixels, being fed into a KNN model that measured the relations between pixels and their trajectories, then comparing them to known trajectories of pixels related to pixel trajectories in abnormal events.

[3] Maqsood, R., Bajwa, U. I., Saleem, G., Raza, R. H., & Anwar, M. W. (2021). Anomaly recognition from surveillance videos using 3D convolution neural network. Multimedia Tools and Applications, 80(12), 18693–18716.

Using a limited number of frames to create a 3D cube for the training of a 3DConvNet model. From the model, spatial information about the frames could be extracted to then perform a prediction of the classification of which event is occurring in the frames. This methodology tries to allow for less frames to be required, but still struggles with the multi-classification. Improvements in the accuracy of the models could allow for improved detection of different anomalous events.

[4] Hussain, A., Khan, N., Khan, Z. A., Yar, H., & Baik, S. W. (2025). Edge-assisted framework for instant anomaly detection and cloud-based anomaly recognition in smart surveillance. Engineering Applications of Artificial Intelligence, 160, Article 111936.

In this paper, they seek to rapidly identify anomalies using resource-constrained edge devices. A NASNet-Mobile model is used for the classification of anomalous or non-anomalous cases.. Anomalous frames are kept in the cloud. In the cloud the frames are subject to a more detailed classification of their anomalies using ConvNeXtLarge model for higher level feature extraction, along with spatial and temporal features extraction. With a Multi-Scale Feature Selection included, the ConvNextLarge model can perform more specified classifications at a lower cost and faster processing in the cloud.

[5] Nguyen, K.-T., Dinh, D.-T., Do, M. N., & Tran, M.-T. (2020). Anomaly detection in traffic surveillance videos with GAN-based future frame prediction. Proceedings of the International Conference on Multimedia Retrieval, 457–463.

Using a GAN-based approach, frame predictions can be carried out to reduce load data processed, while trying to retain the appropriate accuracy for anomaly detection. Motion detection, Scaled Intensity Loss for the identification of edges in fast moving objects, and the tracking of multiple anomalies are key improvements to surveillance, provided by this paper. The isolation and detection of these features allow for better traffic incident identification.

Chapter 2. Project Justification

In recent years, affordable, high-definition cameras have become common in homes and neighborhoods. Yet, most of these systems remain **passive recorders**—they capture hours of video but only notify homeowners after an incident has already happened. A fire may already be spreading, or a prowler may already have left the property, by the time anyone realizes. This reactive model leaves families with more data but little **real protection**.

The justification for this project lies in its ability to transform surveillance into a **proactive, intelligent assistant** that helps people feel safer and respond faster. Instead of overwhelming users with false alarms triggered by pets, passing cars, or swaying trees, the system will **understand context** and **focus attention where it matters most**.

Two capabilities make this possible:

- **Contextual Entity Recognition**: The system can distinguish between "known" faces and pets versus unfamiliar individuals. A child playing in the yard during the afternoon should not generate an alert—but a stranger entering the same yard at 3:00 AM should trigger an immediate notification. This **human-centered intelligence** drastically reduces false positives and restores trust in the alerts.

- **Multi-Class Anomaly Detection**: Beyond presence detection, specialized AI models can identify **critical, high-stakes events**—such as fires, violent altercations, robberies, weapons, crashes, or falls. Instead of vague "motion detected" warnings, the system provides **clear, actionable alerts** that empower families to call for help or take immediate action.

What makes this project especially timely is the convergence of two trends: the **growing accessibility of powerful AI tools** (like YOLOv8, TensorRT, and cloud-native platforms) and the **widespread adoption of connected cameras** in homes. Academic research has already demonstrated the effectiveness of these techniques individually, while industry tools now make real-time, large-scale deployment feasible for small teams. This is the right moment to combine them into a **holistic, adaptive, and affordable security solution**.

Ultimately, the project aims to give homeowners **peace of mind**. Instead of acting as a silent observer, the system becomes an **active partner in safety**—one that learns the rhythms of a household, reduces noise, and ensures that when an alert comes through, it truly matters. By delivering not just more video, but **timely, actionable intelligence**, the project promises a tangible improvement in both security and everyday quality of life.

Chapter 3. Project Requirements

Requirements are expressed as **user stories**, grouped into **Essential**, **Desired**, and **Optional** features, with **Non-Functional Requirements** ensuring reliability, security, and usability.

Functional Requirements

1. Essential (MVP)

- **User Authentication & Management**
  - *As a user, I want to create/login to my account securely, and update details when needed, so my system stays private and safe.*

- **Camera Management**
  - *As a user, I want to register my cameras, so their feeds can be analyzed.*

- **Core Detection & Alerts**
  - *As a user, I want real-time detection of fire, robbery, and violence, and instant alerts with type, timestamp, and screenshot, so I can act quickly.*

- **Incident Logs & Review**
  - *As a user, I want incidents stored and visible on a dashboard, so I can review history or share with authorities.*

2. Desired (Enhancements)

- **Entity Recognition**
  - *As a user, I want the system to recognize family/pets and alert me only for strangers or odd activity, so I avoid false alarms.*

- **Expanded Detection**
  - *As a user, I want detection of crashes and falls, so I can assist in accidents or care for elderly relatives.*

- **Reporting**
  - *As a user, I want charts and summaries, so I can see patterns at a glance.*

3. Optional (Future Scope)

- **Behavioral Analysis**

    - *As a user, I want the system to learn normal routines, so subtle anomalies stand out.*

- **Audio Detection**
    - *As a user, I want detection of sounds like glass breaking or alarms, so I gain extra protection.*

- **Multi-Modal Alerts**
    - *As a user, I want to choose SMS, email, or call by severity, so I never miss critical alerts.*

Non-Functional Requirements

- **Performance**: Alerts under 60s; process ≥1 frame/3s per stream.
- **Accuracy**: ≥85% precision and recall for core models.
- **Security**: Cognito/RBAC for accounts; TLS 1.2+ and AES-256 encryption.
- **Data**: Store only incident-related video; discard non-events.
- **Scalability**: Cloud backend must scale horizontally.
- **Usability**: Dashboard must be clean, intuitive, and responsive.

Chapter 4. Dependencies and Deliverables

Dependencies

The project depends on several external factors that are critical to its success. If any of these are missing, progress would come to a complete halt.

- **Cloud Platform Access**: Access to a major provider such as AWS, GCP, or Azure is required to provision GPU-enabled compute, scalable storage, managed databases, and AI/ML services.

- **High-Quality Training Datasets**: The models rely on large, diverse, and accurately labeled datasets for anomalies (e.g., fire, violence, crashes). Without them, the system cannot achieve the target accuracy of 85%.

- **Prototyping Hardware**: At least one IP camera supporting RTSP is essential for real-world video ingestion tests and end-to-end system validation.

Deliverables

The project will produce a set of tangible outputs that go beyond routine assignments and directly reflect the system's functionality.

- **Cloud-Hosted Backend Application**: A deployed set of microservices handling video ingestion, anomaly detection, incident orchestration, data storage, and notifications.

- **Web-Based Frontend Application**: A functional and secure interface for system management, dashboards, and reviewing detected incidents.

- **Custom-Trained Anomaly Detection Models**: Validated deep learning models packaged for deployment, capable of identifying anomalies and registered entities.

- **System Documentation**: A complete record of the architecture, backend services, APIs, deployment process, and model performance results.

Chapter 5. Project Architecture

Architecture Overview

At a high level, the system flows as follows:

1. **Edge Ingestion** – IP cameras capture video and stream it securely to the cloud. The edge device is lightweight, performing only encoding and transmission. *(Technology: IP Cameras w/ RTSP, AWS Kinesis Video Streams Producer SDK)*

2. **Cloud Ingestion Gateway** – The secure entry point that authenticates streams and routes them to the processing pipeline. *(Technology: AWS Kinesis Video Streams)*

3. **Frame Processing Service** – Extracts video frames (e.g., 1 every 3s) and publishes them as events for downstream processing. *(Technology: AWS Lambda, FFmpeg/OpenCV)*

4. **AI Inference Microservices** – Independent services, each running a specialized model (YOLOv8 for entity recognition, fire/smoke detection, violence/weapon detection, crash/fall detection). They analyze frames in parallel for efficiency. *(Technology: Docker containers on SageMaker Serverless / AWS Fargate / GPU-enabled EC2 with DeepStream)*

5. **Orchestration Service** – Aggregates inference results, applies business logic (e.g., duplicate suppression, entity recognition checks), and confirms incidents. *(Technology: AWS Step Functions or Lambda)*
   zzzz
6. **Data Stores** – Securely store metadata, user info, and incident media. *(Technology: DynamoDB for metadata, S3 for encrypted media, Cognito for user profiles)*

7. **Notification Service** – Sends alerts (push, SMS, or email) to users when incidents are confirmed. *(Technology: Amazon SNS)*

8. **Backend API Gateway** – Provides a secure, versioned API for the frontend to interact with incidents, cameras, and settings. *(Technology: API Gateway + Cognito for JWT-based auth)*

9. **Frontend Web Application** – A modern, responsive web app where users can log in, view real-time alerts, explore past incidents, and manage profiles/devices. *(Technology: React, hosted on S3 + CloudFront)*

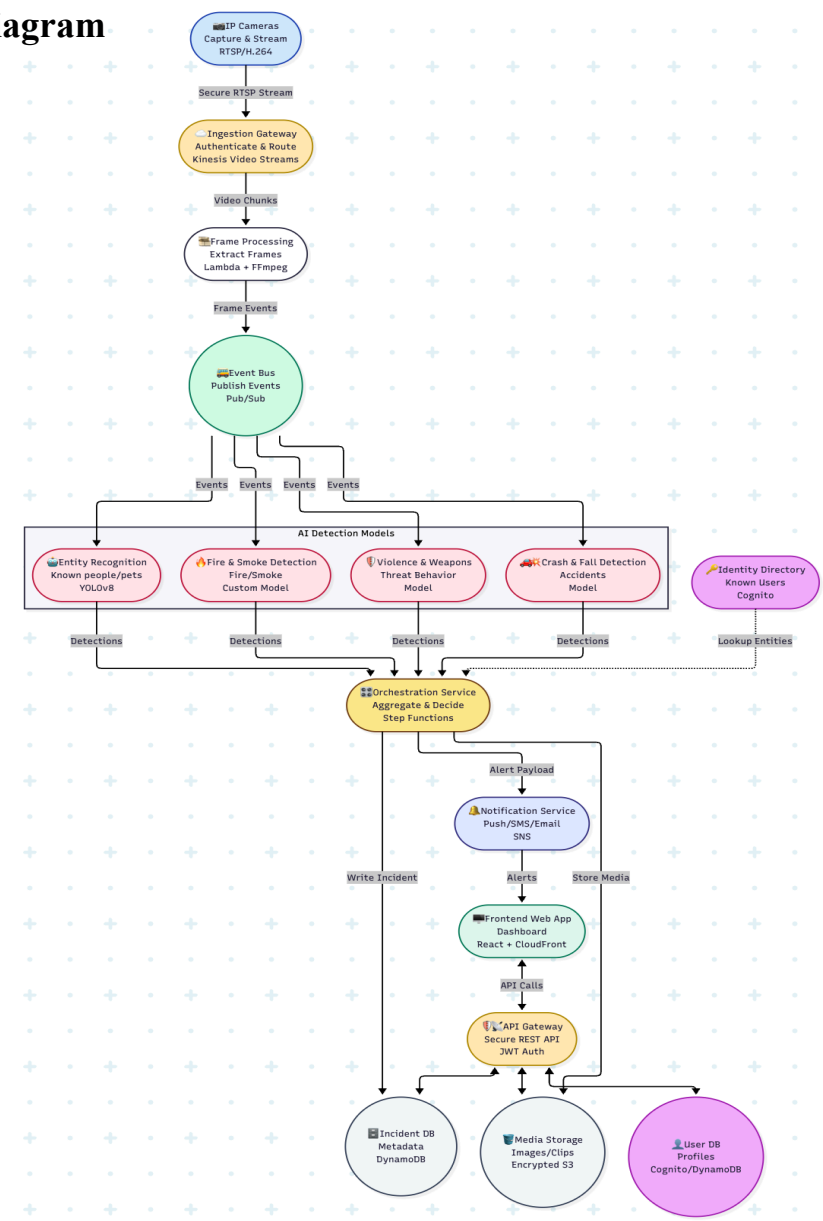
Why Microservices?

The system is built as a collection of **independent services** instead of one big block. This makes it:

- **Specialized**: each anomaly type (fire, fall, violence) gets its own best-fit model.
- **Flexible**: services can be updated or replaced independently without breaking the whole system.
- **Scalable**: if one detection (e.g., crash detection) needs more power, only that service can be scaled up.

This approach keeps the system **adaptable, maintainable, and future-proof**, ready to grow as new anomaly types or user needs arise.

Diagram



Chapter 6. Project Design[

Describe your project design. Types of design artifacts that you provide might include:

- UML diagrams (class diagrams, sequence diagrams, etc.)
- UI Mockups
- Database Entity diagrams
- Hardware block diagrams
- Other artifacts you deem necessary to describe your project in detail.

CMPE Students may want to consider the following alternatives:

- Hardware Block Diagram
- System Block Diagram (Detailed block diagram for each block)
- Circuit Diagram (as applicable)

Evaluated: Workbook Assignment 2

]

Chapter 7. QA, Performance, Deployment Plan

[

Describe the testing, performance evaluation, and deployment plan for your project

CMPE students may want to consider the following alternatives

- Testing: Different tests done or to be done on the project and their outcomes
- Algorithms
- Bill of materials

Evaluated: Workbook Assignment 2

]

Chapter 8. Implementation Plan and Progress

The project is still in its early stages, so current efforts are focused on setting up the working environment and preparing datasets. The overall plan is structured in phases, with Semester 1 dedicated to building and testing the system locally, and Semester 2 focused on migration to the cloud and full-scale integration.

Phase 1: Initial Setup and Environment Configuration (Weeks 1–3, Ongoing)

The team has begun by establishing the foundations needed to start development.

- **Programming Environment Setup**: Installed required libraries and frameworks (Python, PyTorch, OpenCV, YOLOv8). Each team member is standardizing their local development setup.

- **Version Control and Collaboration**: A GitHub repository has been created, and basic branching and review workflows are in place.

- **Exploring Data and Tools**: Initial work involves identifying anomaly datasets (fire, violence, crash) and experimenting with data preprocessing techniques. The team is also running small test scripts to validate compatibility with the chosen frameworks.

Phase 2: Dataset Preparation and Early Model Work (Weeks 4–8)

Once the environment is stable, the focus will shift toward data handling and basic model experimentation.

- **Dataset Acquisition**: Gather publicly available datasets relevant to anomalies and entity recognition.

- **Data Cleaning & Augmentation**: Apply preprocessing steps such as resizing, normalization, and augmentation (rotation, scaling, brightness changes).

- **Preliminary Model Training**: Begin fine-tuning a pre-trained YOLOv8 model on a small subset of the data to validate workflows and understand hyperparameter effects.

Phase 3: Backend and Local Pipeline Development (Semester 1, Weeks 7–11)

The next step will be to connect the trained models with a working local pipeline.

- **Local Prototype Pipeline**: Develop a simple end-to-end flow that captures video (from file or webcam), runs anomaly detection locally, and logs results.
- **Basic Backend Services**: Implement a lightweight local service to manage detections and metadata.
- **API Drafting**: Define initial API endpoints for future integration, keeping them local at this stage.

Phase 4: Cloud Migration and Integration (Semester 2)

With the local prototype validated in Semester 1, Semester 2 will focus on scaling the system into the cloud.

- **Cloud Infrastructure Setup**: Deploy core services (databases, APIs, storage) on AWS.
- **Model Deployment**: Package trained models into containers and deploy them to inference endpoints.
- **Frontend and Integration**: Build a React-based web interface, connect it to the backend APIs, and test with real camera streams.
- **Final Documentation and Testing**: Conduct end-to-end tests, document the full system, and prepare the final report.

Current Progress

At present, the team is in **Phase 1**, actively:

- Configuring local environments,
- Setting up source control and collaboration workflows,
- Exploring anomaly detection datasets, and
- Running small-scale test scripts to ensure tools and libraries work correctly.

Chapter 9. Project Schedule

Phase 1: Initial Setup and Environment Configuration (Weeks 1–3)

Task	Owner	Details
Programming Environment Setup	Dev, Harsh, Cameron	Install Python, PyTorch, OpenCV, YOLOv8. Standardize local environments.
Version Control & Collaboration	Harsh	GitHub repo, branching policy, CI basics.
Exploring Data & Tools	Cameron	Identify anomaly datasets (fire, violence, crash), run test scripts.

Phase 2: Dataset Preparation & Early Model Work (Weeks 4–8)

Task	Owner	Details
Dataset Acquisition	Cameron	Gather datasets for anomalies + entity recognition.
Data Cleaning & Augmentation	Harsh	Preprocess (resize, normalize, augment).
Preliminary Entity Recognition Model	Dev	Fine-tune YOLOv8 on a small dataset subset.
Preliminary Anomaly Detection Models	Dev	Begin training fire/violence/crash detection models.
Workflow Validation	Cameron	Ensure models can run locally, verify dependencies.

Phase 3: Backend and Local Pipeline Development (Weeks 7–11)

Task	Owner	Details
Local Prototype Pipeline	Harsh	Video → detection → logs (end-to-end local).
Basic Backend Services	Dev	Lightweight local service for detections + metadata.
API Drafting (Local)	Cameron	Define draft API endpoints for integration.

Phase 4: Cloud Migration & Full Integration (Semester 2)

Task	Owner	Details
Cloud Infrastructure Setup	Dev	Deploy AWS core services (DBs, APIs, storage).
Model Deployment	Harsh, Cameron	Package trained models (ONNX/TensorRT, Docker) → SageMaker endpoints.
Frontend Web Application	Cameron	Build React frontend, connect APIs.
End-to-End Integration	Dev, Harsh, Cameron	Connect live camera → cloud pipeline → frontend.
Final Documentation & Testing	Dev, Harsh, Cameron	Full testing, final documentation + report.